

MS 3 – Konfiguration der Softwareentwicklung und Qualitätsplanung der Liefergegenstände

Projekt: Lernzeit-Manager

Kurs: ISEF01 – Projekt Software Engineering

Team: Elias Ebertshäuser (Product Owner), Assis Ramadan (Developer, Schwerpunkt Coding), Julian Wagner (Developer, Schwerpunkt Infrastruktur/Deployment/Testing)

Datum: August 2026

1. Konfiguration der Softwareentwicklung

1.1 Anforderungsmanagement

Arbeitsergebnisse:

Die funktionalen Anforderungen sind in [docs/01_Funktionale_Anforderungen.md](#) dokumentiert und wurden im Kickoff-Meeting vom Team abgenommen. Die Qualitätsanforderungen befinden sich in [docs/02_Qualitätsanforderungen.md](#), die Randbedingungen in [docs/03_Randbedingungen.md](#). Diese Dokumente gelten als eingefrorener Anforderungsstand für MS4.

Techniken:

Anforderungen wurden als User-Story-ähnliche Aussagen formuliert, mit eindeutiger ID (z. B. FR-1.1), Priorität (Must/Should/Could) und geschätztem Aufwand in Tagen. Änderungen an Anforderungen werden im Team per Weekly-Meeting besprochen und müssen von allen drei Mitgliedern akzeptiert werden, bevor sie in das Dokument einfließen.

Qualitätssicherung:

Jede Anforderung wird gegen die Kriterien SMART (Spezifisch, Messbar, Erreichbar, Relevant, Terminiert) geprüft. Unklare Anforderungen werden im Team-Meeting geklärt, bevor mit der Implementierung begonnen wird. Testfälle werden direkt aus den Must-Anforderungen abgeleitet (Traceability).

1.2 Artefakte und Zwischenergebnisse

Im Projektverlauf werden folgende Artefakte erstellt und gepflegt:

Anforderungsliste:

[docs/01_Funktionale_Anforderungen.md](#) – enthält alle funktionalen Anforderungen mit Priorität, ID und Aufwandsschätzung. Eingefrorener Stand seit Kickoff.

Architekturbeschreibung:

Bestandteil der technischen Dokumentation in MS4. Beschreibt die Systemarchitektur als REST-API-Architektur mit Angular-SPA im Frontend, Flask-Backend und PostgreSQL-Datenbank. Wird als Diagramm (draw.io) und Fließtext erstellt.

Product Backlog:

Geführt in GitHub Projects. Enthält alle Must-Anforderungen als Issues, priorisiert nach MS4-Lieferziel. Jedes

Issue referenziert die zugehörige FR-Nummer.

Sprint Backlog / Feature-Tasks:

Pro Entwicklungsiteration werden Issues aus dem Product Backlog auf Entwickler verteilt. Tracking erfolgt über GitHub Projects Board (Spalten: To Do → In Progress → In Review → Done).

Bugliste:

Bugs werden als GitHub Issues mit Label „bug“ erfasst. Kritische Bugs (Funktionsverlust) werden vor dem nächsten Merge auf `main` behoben.

Testfälle und Testprotokoll:

Backend-Testfälle als pytest-Dateien unter `backend/tests/`. Frontend-Testfälle als Vitest-Dateien unter `frontend/src/`. Das Testprotokoll wird für MS4 als Dokument erstellt und enthält: Testfall-ID, Beschreibung, erwartetes Ergebnis, tatsächliches Ergebnis, Status (bestanden/fehlgeschlagen).

ExecPlans:

Für jede größere Änderung wird vorab ein Execution Plan erstellt (`docs/ExecPlans/active/`). Das ExecPlan-Format ist in `docs/PLANS.md` spezifiziert. Abgeschlossene Pläne werden nach `docs/ExecPlans/completed/` verschoben.

1.3 Eingesetzte Programmiersprachen

Bereich	Sprache	Version
Backend	Python	3.12
Frontend	TypeScript	5.x (via Angular CLI 22)
Datenbankmigrationen	SQL (via Alembic/Flask-Migrate)	–
Containerisierung	YAML (Docker Compose)	–
CI/CD	YAML (GitHub Actions)	–

1.4 Systeme für Organisation, Entwicklung, Test, Bereitstellung und Betrieb

Organisation:

Zweck	System
Offizielle Meilensteinverwaltung	Redmine (redmine-se.iubh.de)
Technisches Aufgaben-Board	GitHub Projects
Kommunikation	MS Teams (wöchentliches Meeting mittwochs ca. 15:30 Uhr)
Dokumentenablage (intern)	OneDrive

Entwicklung:

Zweck	System
Versionskontrolle	Git / GitHub (privates Repository)
Backend-Framework	Flask 3.1 (Python)
Frontend-Framework	Angular 22 (TypeScript)
ORM / Migrationen	Flask-SQLAlchemy + Flask-Migrate (Alembic)
Authentifizierung	Flask-JWT-Extended (geplant für MS4)
CORS	Flask-CORS
Lokale Datenbank	PostgreSQL 16 via Docker Desktop
Empfohlene IDE	VS Code mit Extensions (Angular Language Service, Python, Pylance, Ruff, ESLint, Docker)

Test:

Zweck	System
Backend Unit-Tests	pytest + pytest-flask
Backend-Linting	ruff (Regeln: E, F, W, I)
Frontend Unit-Tests	Vitest (Angular CLI 22 Standard)
Frontend-Linting	ESLint via angular-eslint
CI-Ausführung	GitHub Actions (automatisch bei jedem Push)
Geplant (MS4)	Playwright für End-to-End-Tests

Bereitstellung:

Zweck	System
Hosting (Backend + Frontend)	Railway
Datenbank (Produktion)	PostgreSQL via Railway Add-on
CI/CD-Pipeline	GitHub Actions → automatischer Deploy bei Push auf main

Betrieb:

Zweck	System
Log-Einsicht	railway logs CLI
Datenbank-GUI (lokal)	pgAdmin 4
Secrets-Verwaltung	Railway Environment Variables (nie im Repository)

1.5 Rollen und Ergebnisverantwortung

Rolle	Person	Verantwortung
Product Owner	Elias Ebertshäuser	Anforderungen priorisieren, Backlog pflegen, Abnahme von Features, Kommunikation mit Tutor
Developer (Coding)	Assis Ramadan	Backend-Implementierung (Flask-API, Datenbankmodelle, Tests), Frontend-Komponenten
Developer (Infrastruktur)	Julian Wagner	CI/CD-Pipeline, Docker-Setup, Railway-Deployment, Testing-Strategie, Code-Reviews

Alle Teammitglieder sind gemeinsam verantwortlich für:

- Einhaltung der Code-Qualitätsstandards (ruff, ESLint)
- Review von Pull Requests (mindestens ein Reviewer pro PR)
- Rechtzeitige Lieferung der Meilensteine in Redmine

1.6 Vorgehensmodell

Das Team verwendet ein **schlankes iteratives Vorgehen** angelehnt an Scrum, angepasst an die Größe und den Umfang des Projekts.

Iterationsrhythmus:

Entwicklung in 1–2-wöchigen Iterationen. Jede Iteration endet mit einem lauffähigen, getesteten Inkrement, das auf **main** gemergt ist.

Ablauf einer Iteration:

1. **Planung:** Aus dem Product Backlog werden Issues für die Iteration ausgewählt und auf Entwickler verteilt (GitHub Projects Board).
2. **Entwicklung:** Jeder Entwickler arbeitet auf einem eigenen Feature-Branch (**feature/<beschreibung>** oder **fix/<beschreibung>**). Vor der Implementierung einer größeren Änderung wird ein ExecPlan erstellt.
3. **Review:** Fertige Features werden als Pull Request auf GitHub eingereicht. Mindestens ein anderes Teammitglied reviewed den Code. Die CI-Pipeline muss grün sein, bevor gemergt wird.
4. **Merge:** Nach Approval und grüner CI wird der Branch auf **main** gemergt. **main** ist jederzeit deploybar.
5. **Retrospektive:** Im Weekly-Meeting kurze Reflexion: Was lief gut? Was wird angepasst?

Branch-Konventionen:

- **main** – stabil, immer deploybar, kein direkter Push
- **feature/<kurzbeschreibung>** – neue Funktionalität
- **fix/<kurzbeschreibung>** – Bugfixes
- **docs/<kurzbeschreibung>** – Dokumentationsänderungen

Commit-Konventionen:

Commit-Messages auf Deutsch oder Englisch im Imperativ, mit Bezug auf FR-Nummer wenn möglich (Beispiel: **Timer: Start/Pause/Stop implementiert (FR-4.1)**).

2. Konfiguration der Liefergegenstände

2.1 Übersicht aller Liefergegenstände

Meilenstein	Liefergegenstand	Format	Ablageort
MS 0	Meilensteinplan	PDF oder Bild	Redmine
MS 1	Projektkonfiguration	PDF	Redmine
MS 2	Projektvideo (max. 5 Min.)	Video / Link	Redmine
MS 3	Konfiguration SW-Entwicklung + Qualitätsplanung	PDF	Redmine
MS 4	Softwaresystem + vollständige Dokumentation	Mehrere Dokumente + Link	Redmine
MS 5	Ergebnispräsentation (max. 20 Min.)	Video / Link	Redmine
MS 6	Projektbericht	PDF via Turnitin	myCampus / Turnitin

2.2 MS 4 – Liefergegenstände im Detail

MS 4 ist der umfangreichste Meilenstein. Folgende Dokumente werden erstellt:

Benutzerhandbuch:

Beschreibt alle Funktionen der Anwendung aus Nutzerperspektive. Enthält Screenshots und Schritt-für-Schritt-Anleitungen für jeden Anwendungsfall (FR-1 bis FR-6). Umfang: ca. 10–15 Seiten.

Fachliche Dokumentation:

Beschreibt fachliche Prozesse (z. B. Ablauf einer Lernsession), Geschäftsregeln (z. B. Berechnung des Workloads aus ECTS) und fachliche Konzepte (z. B. Lernziel-Lebenszyklus).

Technische Dokumentation:

Enthält Architekturübersicht (Komponentendiagramm), API-Beschreibung (alle Endpunkte mit Request/Response), Datenbankschema (Entity-Relationship-Diagramm), Beschreibung der Deployment-Infrastruktur (Railway).

Betriebsdokumentation:

Beschreibt Installation und Konfiguration der Anwendung, Admin-Zugangsdaten für Tutor-Review, Beschreibung der Umgebungsvariablen.

Testabschlussbericht:

Enthält alle Testfälle (ID, Beschreibung, Vorbedingung, Schritte, erwartetes Ergebnis), Testprotokolle (Datum, Ergebnis, Tester), Ergebnis-Zusammenfassung.

Programmcode:

Link zum GitHub-Repository (Cooke0/Projekt-Lernzeit_Manager).

Link zum System:

URL der auf Railway deployten Anwendung.

Test-Accounts:

Liste mit Zugangsdaten für den Tutor (keine echten personenbezogenen Daten, nur Testdaten).

2.3 MS 6 – Projektbericht

3er-Gruppe: 21–30 Seiten Textteil. Jedes Mitglied verfasst einen zusammenhängenden Abschnitt von 7–10 Seiten, klar zugeordnet auf dem Titelblatt. Abgabe einzeln über myCampus/Turnitin mit Dateiname

JJJJMMTT_Nachname_Vorname_Matrikelnummer_ISEF01.

3. Qualitätsplanung

3.1 Qualitätsziele

Qualitätsziel	Beschreibung	Messbar durch
Korrektheit	Alle Must-Anforderungen (FR-x.y mit Priorität M) sind korrekt implementiert	Alle Testfälle für Must-Anforderungen bestehen
Wartbarkeit	Code ist lesbar, einheitlich formatiert, ohne Toter Code	ruff check / ESLint ohne Fehler; kein auskommentierter Code
Zuverlässigkeit	Anwendung ist im Browser des Tutors ohne Installation nutzbar	Erfolgreiches Railway-Deployment; Tutor-Testrunde ohne Absturz
Datenschutz	Keine echten personenbezogenen Daten in Testdaten oder im Repository	Code-Review; .env nie committet; Test-Accounts nur mit Dummy-Daten
Testbarkeit	Backend und Frontend sind automatisch testbar	CI-Pipeline grün; Testabdeckung der Must-Features vollständig

3.2 Qualität des Softwaresystems

Statische Qualitätssicherung – Anforderungen:

Anforderungen werden gegen SMART-Kriterien geprüft. Jede Must-Anforderung erhält mindestens einen Testfall, bevor mit der Implementierung begonnen wird (Test-First-Ansatz wo möglich).

Statische Qualitätssicherung – Programmcode:

Maßnahme	Tool	Ausführung	Verantwortung
Python-Linting und Import-Sortierung	ruff (Regeln E, F, W, I)	automatisch in CI bei jedem Push	Julian
TypeScript-Linting	ESLint via angular-eslint	automatisch in CI bei jedem Push	Julian

Maßnahme	Tool	Ausführung	Verantwortung
Code-Review	GitHub Pull Request Review	manuell, vor jedem Merge auf main	alle

Dynamische Qualitätssicherung – Tests:

Teststufe	Beschreibung	Tool	Wann
Backend Unit-Tests	Testen einzelne Flask-Endpunkte isoliert (SQLite in-memory)	pytest + pytest-flask	automatisch in CI; lokal vor jedem Push
Frontend Unit-Tests	Testen Angular-Komponenten und Services isoliert	Vitest	automatisch in CI; lokal vor jedem Push
End-to-End-Tests (geplant)	Simulieren vollständige User-Journeys im Browser	Playwright	manuell vor MS4-Abgabe
Manueller Systemtest	Tutor-Szenario durchspielen (alle Must-Features)	–	vor MS4-Abgabe durch Assis

Architekturentscheidungen – Qualitätssicherung:

Architekturentscheidungen werden in ExecPlans dokumentiert (Decision Log). Abweichungen vom ursprünglichen Design werden im Decision Log begründet. Kein unüberprüfter Code auf **main** (CI-Pflicht).

IT-Sicherheit:

- Passwörter werden ausschließlich gehasht gespeichert (bcrypt via Flask-Bcrypt oder Werkzeug).
- Authentifizierung über JWT-Token (Flask-JWT-Extended); Token-Laufzeit begrenzt.
- Secrets (Datenbankpasswort, JWT-Secret) ausschließlich über Umgebungsvariablen; nie im Repository.
- HTTPS in der Produktionsumgebung (Railway erzwingt HTTPS automatisch).
- SQL-Injection-Schutz durch SQLAlchemy ORM (kein rohes SQL mit Nutzereingaben).
- CORS eingeschränkt auf bekannte Ursprünge (lokale Entwicklung: **localhost:4200**, Produktion: Railway-Domain).

Datenschutz (DSGVO):

- Keine echten personenbezogenen Daten in Testdaten, Prompts oder im Repository.
- Test-Accounts enthalten ausschließlich fiktive Daten.
- Die Anwendung speichert nur die Daten, die der Nutzer explizit eingibt (Lernziele, Lernsessions).

3.3 Qualität der Liefergegenstände

Liefergegenstand	Qualitätssicherungsmaßnahme	Verantwortung
Projektkonfiguration (MS1)	Vollständigkeitsprüfung gegen IU-Anforderungsliste; Gegenlesen durch alle Mitglieder	Elias Ebertshäuser
Projektvideo (MS2)	Inhaltliche Prüfung: Alle Pflichtpunkte abgedeckt; Länge max. 5 Min.	Alle

Liefergegenstand	Qualitätssicherungsmaßnahme	Verantwortung
Dieses Dokument (MS3)	Vollständigkeitsprüfung gegen IU-Anforderungsliste; Gegenlesen durch alle	Julian Wagner
Benutzerhandbuch (MS4)	Testrunde: Tutor-Szenario anhand Handbuch durchgespielt	Assis Ramadan
Technische Doku (MS4)	Gegenlesen durch anderen Entwickler; Prüfung Aktualität gegenüber Code	Julian Wagner
Testabschlussbericht (MS4)	Alle Must-Testfälle vorhanden; Protokoll vollständig ausgefüllt	Julian Wagner
Präsentationsvideo (MS5)	Inhaltliche Prüfung: Alle Pflichtpunkte; Länge max. 20 Min.; Demo lauffähig	Alle
Projektbericht (MS6)	Turnitin-Plagiatsprüfung; Seitenumfang je Mitglied 7–10 Seiten; Gegenlesen	Alle

Erstellt: August 2026 – Team Lernzeit-Manager (ISEF01)